



আন্তর্জাতিক ইসলামী বিশ্ববিদ্যালয় চট্টগ্রাম
الجامعة الإسلامية العالمية شيتاغونغ
International Islamic University Chittagong

Department of Electrical and Electronic Engineering

Project Report

Project Title : Speech Enhancement Using Spectral Subtraction with FIR and IIR Filters.

Course Code : EEE – 3604

Course Title : Digital Signal Processing Sessional

Submitted To : **MOHAMMAD ABDUL KADER**

Associate Professor

Dept. of EEE, IIUC

Submitted By

ID & Name :	ET233042	Shahidul Islam
	ET233053	Md.Nazmul Hossan Talukder Fahim
	ET233073	Mohammed Nazrul Islam

Section : 6B

Session : Spring – 26

Date of Submission : 29.06.2026

Remark:

2. Objectives:

1. To understand the theoretical basis of Finite Impulse Response (FIR) and Infinite Impulse Response (IIR) filter design.
2. To learn the Short-Time Fourier Transform (STFT)-based Spectral Subtraction algorithm for suppression of additive background noise in real speech signals.
3. To design a simple and effective speech enhancement system.
4. To perform a complete complex engineering investigation including problem formulation, design, implementation, and critical analysis

3. Formulation of the Problem:

3.1 Project Description: Speech is recorded in a wide variety of real-world environments, each introducing a distinct category of background noise. Understanding these noise types and their spectral behavior is the foundational step in designing any enhancement system.

Home environments introduce electrical mains hum (50 Hz), Ceiling fan hum, and refrigerator motor noise. These are largely stationary; their spectral characteristics remain relatively constant over time. Making them well-suited to frequency-domain suppression. However, home environments also produce non-stationary interference: televisions and background music carry speech-like spectra, keyboard keystrokes and door slams generate short impulsive bursts, and sounds such as dog barking or a baby crying appear as unpredictable energy bursts across wide frequency bands.

Outdoor environments are dominated by wind noise; a low-frequency, spectrally shaped disturbance with the majority of energy below 100 Hz. Along with ambient crowd murmur and light rain, which are approximately stationary broadband noise sources. Sudden transient events such as bird calls are non-stationary and cannot be predicted from any initial noise estimate.

Traffic environments produce two distinct noise profiles: (i) constant highway or engine rumble; stationary, low-to-mid frequency broadband noise, which is the ideal target for spectral-domain suppression; and (ii) time-varying events such as passing vehicles, car horns, and emergency sirens, which are non-stationary and change amplitude rapidly.

A critical engineering observation emerges: the noise types most amenable to DSP suppression are those that are stationary, additive, and broadband, specifically constant fan hum, electrical mains hum, steady rain, and highway engine rumble. Non-stationary events such as speech-

like backgrounds, impulsive sounds, and time-varying sources fundamentally violate the stationarity assumption of the spectral subtraction algorithm and cannot be reliably suppressed by this system. This boundary defines the precise scope of the engineering problem.

Problem Statement: *Given a speech recording contaminated by stationary additive background noise, such as fan hum, electrical mains interference, or constant highway/traffic rumble. Design and implement a multi-stage DSP pipeline that (i) isolates the speech frequency band, (ii) estimates and suppresses the stationary noise component using spectral subtraction without a reference noise channel, and (iii) smooths residual artifacts all without requiring prior knowledge of the noise type.*

3.2 Specification/Needs:

Parameter	Value
Input Signal	Speech audio file in “.wav” Format
Target Noise	System handles stationary, additive broadband noise only
Noise Reference	None, Noise estimated from initial silence of the signal
Frequency Range Preserved	Standard speech band (100 Hz – 4000 Hz)
Output Format	Three WAV file
Implementation Language	Python with Numpy, SciPy and Matplotlib

3.3 Constraints:

Constraint	Description
Non-Stationary Noise	Strong TV audio, car horns, impulsive bursts, and speech-like backgrounds cannot be suppressed.
Offline Processing Only	Not designed for real-time streaming; full signal must be available before processing begins
No Pre-built Filter Functions	scipy.signal and similar filter design libraries are not permitted
No Machine Learning	System must operate without any trained model or external dataset
Noise Estimation Frames	First 12 frames assumed to be background noise/silence. System fails if there is no silence reference at start.

3.4 Complex Engineering Problem Justification

P1: Depth of Knowledge Required

This project requires knowledge of:

- Fourier analysis, convolution, and windowed-sinc filter design (K3: Mathematics)
- Digital filter theory - FIR/IIR design, frequency response, stability (K4: Engineering Fundamentals)
- STFT analysis-synthesis framework and Overlap-Add reconstruction (K5: Engineering Principles)
- Practical implementation - data type handling, numerical stability, parameter tuning (K6: Engineering Practice)

P2: Conflicting Technical Requirements

The project involves several conflicting requirements. Higher noise suppression (α) causes speech distortion; more FIR taps give better roll-off but increase delay; lower spectral floor (β) reduces noise but risks suppressing speech; lower IIR cutoff smooths artifacts but attenuates consonants.

Requirement A	Requirement B	Conflict / Trade-off
High noise suppression (high α in spectral subtraction)	Minimal speech distortion	Higher α removes more noise but over-subtracts speech energy causing musical noise
Sharp filter roll-off (More FIR taps)	Low processing delay and memory	More taps means better stopband rejection but greater group delay (401 taps = 200 samples) and higher convolution cost
Aggressive spectral floor (low β)	Avoid signal suppression	Low β reduces noise floor further but risks completely zeroing bins that contain valid speech energy

Requirement A	Requirement B	Conflict / Trade-off
Aggressive IIR smoothing (low $f_{c_{iir}}$)	Preserve high-frequency consonants	Lowering the IIR cutoff removes fricative consonants critical for intelligibility
Wide noise estimation window (many frames)	Non-stationary noise tracking	Using more initial frames improves noise magnitude estimate accuracy but fails if noise characteristics change across the recording

P3: No Unique Way to Design

Multiple valid approaches exist:

- Wiener filter
- LMS adaptive filter
- Wavelet denoising
- DNN-based enhancement

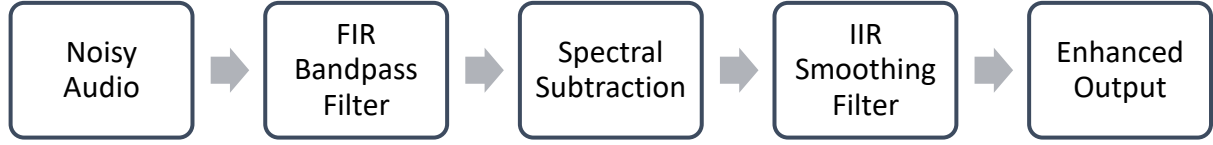
Depth of analysis was required to evaluate each method against the given constraints and justify spectral subtraction as the most appropriate choice.

P4: Familiarity of Issues

The project involves infrequently encountered issues such as musical noise artifact in spectral subtraction, WOLA zero-division at signal edges, and phase preservation during STFT magnitude modification - issues not commonly covered in standard DSP courses.

4. Methodology:

The system processes a single-channel noisy speech WAV file through three sequential DSP stages. The overall pipeline is illustrated below:



4.1 FIR Bandpass Filter: The first stage isolates the speech-relevant frequency band (100 Hz – 4000 Hz) using a linear-phase FIR bandpass filter designed with the windowing method.

The ideal bandpass filter has an impulse response

$$h[0] = \frac{(\omega_2 - \omega_1)}{\pi}, \text{ for } n = 0$$
$$h[n] = \frac{\sin(\omega_2 n) - \sin(\omega_1 n)}{\pi n}, \text{ for } n \neq 0$$

Where $\omega_1 = \frac{2\pi f_{low}}{f_s} = \frac{2\pi 100}{f_s}$, and $\omega_2 = \frac{2\pi f_{high}}{f_s} = \frac{2\pi 4000}{f_s}$.

Since $h[n]$ is infinite-length and non-causal, it is truncated to $M+1 = 401$ taps and multiplied by the Hamming window: $w[n] = 0.54 + 0.46 \cdot \cos\left(\frac{2\pi n}{M}\right)$.

The function `create_fir_bandpass(taps, f_low, f_high, fs)` constructs $h[n]$ analytically without any library-provided filter design routines. The filter is applied by linear convolution using `np.convolve` with `mode='same'` to maintain the original signal length.

4.2 Spectral Subtraction: The spectral subtraction method works in the frequency domain. The core idea is: if we know what the noise spectrum looks like, we can subtract it from the noisy speech spectrum to recover a cleaner signal. It operates frame by frame using the Short-Time Fourier Transform (STFT).

4.2.1 STFT Analysis (Framing)

The filtered signal from Stage 1 is divided into short overlapping frames:

- Frame length: $L = 1024$ samples
- Hop Size: $H = 512$ samples

- Analysis window: Hamming Window $w[n] = 0.54 + 0.46 \cdot \cos\left(\frac{2\pi n}{L-1}\right)$.

Each frame is windowed before taking the FFT: $x_w[n] = x[n] \cdot w[n]$

Then the N-point DFT is applied to each windowed frame: $X(k) = \sum x_w[n] \cdot e^{-j\frac{2\pi kn}{N}}$

The result is decomposed into magnitude and phase: $X(k) = |X(k)| \cdot e^{j\angle X(k)}$, where $|X(k)|$ is the magnitude spectrum and $\angle X(k)$ is the phase spectrum.

4.2.2 Noise Estimation

The noise magnitude spectrum is estimated from the **first 12 frames**, which are assumed to contain only background noise (silence before speech begins):

$$N(k) = \frac{1}{F} \sum X_f(k), f = 0, 1, 2, \dots, 11 \text{ Where } F = 12 \text{ is the number of noise estimation frames.}$$

4.2.3 Magnitude Subtraction with Spectral Floor

For every frame in the signal, the estimated noise magnitude is subtracted from the noisy speech magnitude using an **over-subtraction factor** α : $|\hat{S}(k)| = |X(k)| - \alpha \cdot \hat{N}(k)$

However, this can produce negative values (over-subtraction), which leads to an artifact called **musical noise** - isolated tonal distortions in quiet regions. To prevent this, a **spectral floor** β is applied: $|\hat{S}(k)| = \max(|X(k)| - \alpha \hat{N}(k), \beta |X(k)|)$

This ensures the subtracted magnitude never falls below $\beta |X(k)|$. In this system $\alpha = 2.0$ and $\beta = 0.08$

4.2.4 Phase Preservation and IFFT

The original phase is preserved from the noisy signal. The enhanced spectrum is reconstructed as: $\hat{S}(k) = |\hat{S}(k)| e^{j\angle X(k)}$

The time-domain frame is recovered using the Inverse DFT: $\hat{s}[n] = \frac{1}{N} \sum \hat{S}(k) \cdot e^{j\frac{2\pi kn}{N}}$

4.2.5 Overlap-Add (OLA) Reconstruction

Each reconstructed frame is multiplied again by the synthesis window and added back into the output buffer at its original position (Overlap-Add method):

$$y[n] = \sum \hat{s}_m[n - mH] \cdot w[n - mH]$$

To normalize the output correctly, each sample is divided by the sum of squared window weights accumulated at that position: $y_{norm}[n] = \frac{y[n]}{\sum w^2[n-mH]}$

Any position where the weight sum equals zero is set to 1.0 to prevent division by zero (edge guard).

4.3 IIR Low-Pass Smoothing Filter: After spectral subtraction, residual high-frequency artifacts may remain. A first-order IIR low-pass filter is applied as a final smoothing step. It is the digital equivalent of a simple analog RC low-pass circuit.

4.3.1 Analog Prototype and Discretization

The analog RC low-pass filter has a time constant: $\tau = \frac{1}{2\pi f_c}$, where $f_c = 3500 \text{ Hz}$ is the cutoff frequency.

Using the Backward Euler discretization with sample period $d\tau = \frac{1}{f_s}$: $\alpha_{iir} = \frac{d\tau}{\tau + d\tau}$

Substituting τ : $\alpha_{iir} = \frac{d\tau \cdot 2\pi \cdot f_c}{1 + d\tau \cdot 2\pi \cdot f_c}$

This α_{iir} value determines how much the filter weighs the current input versus the previous output.

4.3.2 Difference Equation

The filter is implemented sample by sample using the first-order IIR difference equation:

$$y[n] = \alpha_{iir} \cdot x[n] + (1 - \alpha_{iir}) \cdot y[n - 1]$$

Where,

- $x[n]$ = current input sample
- $y[n]$ = current output sample
- $y[n - 1]$ = previous output sample
- α_{iir} = smoothing coefficient (computed from f_c and f_s)

4.3.3 Filter Properties

Type: First-order IIR, single real pole at $z = (1 - \alpha_{iir})$. The pole is always inside the unit circle since $0 < \alpha_{iir} < 1$, guaranteeing unconditional stability.

5. Result & Analysis:

To test the system, we have recorded three audios in three different scenarios First one is inside home, second one is Beside Dhaka-Chittagong Highway, and third one is in busy street specifically Bohoddarhat Circle. Let's go through each case.

5.1 Sample 1 – Home Environment: The original signal shows a relatively clean speech waveform with a consistent low amplitude background noise floor visible in the inter-word pauses characteristic of stationary fan noise. We can verify it through listening to the audio.

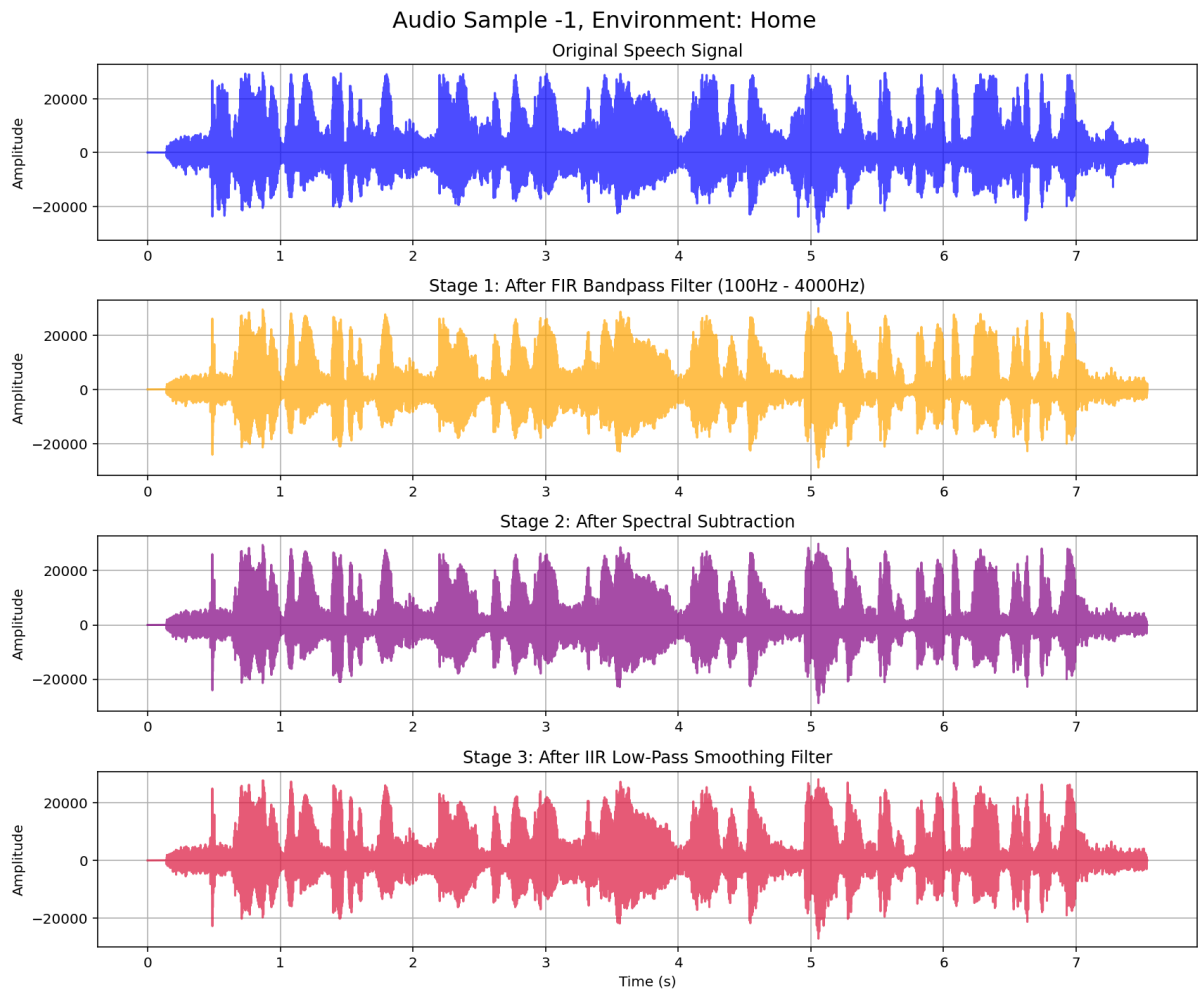


Fig. 1 Original and Filtered Plot of the Sample Speech - 1

Stage 1 (FIR): The waveform shape is largely unchanged after the FIR filter. This is expected ceiling fan noise typically sits within the 100–4000 Hz speech band, so the bandpass filter alone cannot isolate it from speech. The FIR stage contributes minimally here beyond removing any sub-100 Hz rumble from the fan motor.

Stage 2 (Spectral Subtraction): A visible improvement can be observed. The inter word silence gaps, which previously showed a consistent noise floor, are noticeably cleaner. The speech envelope is well preserved. Since fan noise is highly stationary, the noise estimate from the first 12 frames accurately represents the noise across the entire recording this is the ideal operating condition for spectral subtraction.

Stage 3 (IIR): The waveform is slightly smoother compared to Stage 2. No significant structural change is visible at the waveform level, which is expected since the IIR filter operates subtly and its effect is more perceptible in listening than in waveform observation.

Overall: The system performed well on this sample. Fan noise being stationary and broadband matched the core assumption of the spectral subtraction algorithm, resulting in effective noise suppression with minimal speech distortion.

5.2 Sample 2 – Beside Highway: The original signal immediately shows a denser, thicker waveform compared to Sample 1. Even during speech pauses, the amplitude does not approach zero.

Stage 1 (FIR): A slight thinning of the waveform is visible. Highway rumble below 100 Hz is removed, which reduces the low-frequency energy contribution from engine and road noise. However, the mid-frequency highway noise within the speech band still remains.

Stage 2 (Spectral Subtraction): This is the most impactful stage for this sample. The noise floor in the pauses is substantially reduced; The waveform is noticeably thinner in silence regions compared to the original. The speech segments are now more distinctly separated from the background. This result validates the system's effectiveness for stationary highway noise, as the constant highway spectrum is well-estimated from the initial silence frames.

Stage 3 (IIR): Slight amplitude smoothing is observed. The sharp transient peaks are marginally rounded, producing a cleaner final output.

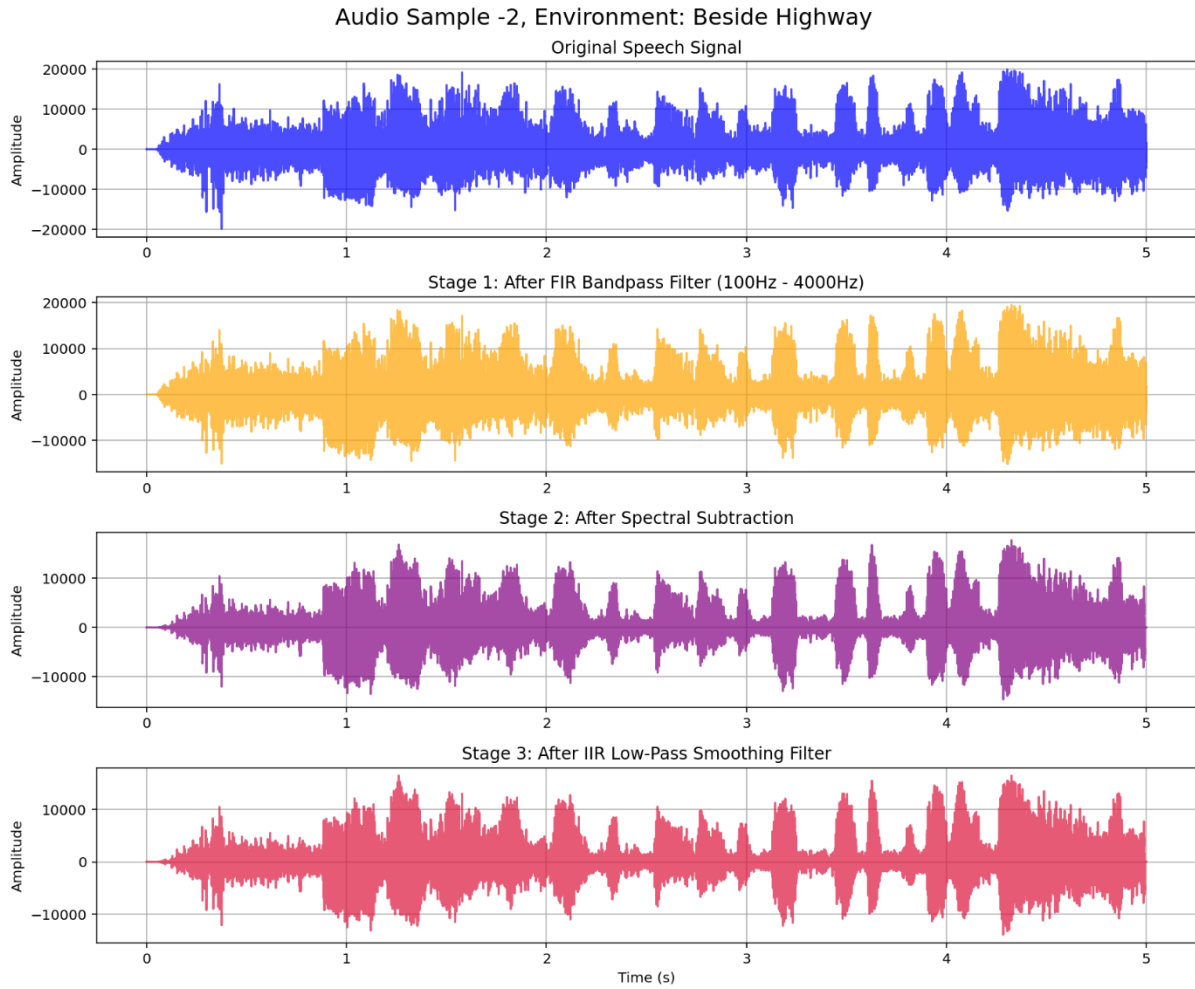


Fig. 2 Original and Filtered Plot of the Sample Speech - 2

Overall: Highway noise, being predominantly stationary and broadband, represents a near-ideal case for this system. The multi-stage pipeline produced a clear reduction in noise floor with good preservation of speech structure. This is arguably the strongest result among the three test cases.

5.3 Sample 3 – Busy Street (Heavy Traffic): This is the most challenging case and the results reveal the fundamental limitation of the system. The original signal shows an unusually large amplitude burst at the start, followed by a dramatically lower amplitude throughout the remainder of the recording. This indicates a highly non-stationary noise environment. Traffic intensity fluctuates significantly across the recording.

Stage 1 (FIR): The initial burst is partially reduced and the overall waveform is somewhat thinned. The FIR filter removes sub 100 Hz Road rumble and high-frequency components, providing modest improvement.

Stage 2 (Spectral Subtraction): The result here reveals the system's core weakness. The noise estimation was performed on the first 12 frames; which in this case captured the loudest traffic burst at the beginning. This large noise estimate was then subtracted from the entire recording, including the quieter later segments where the actual noise level was lower. As a result, the speech energy in the later sections (1–7 seconds) appears partially suppressed rather than enhanced. The system essentially over-subtracts the quieter speech regions. The waveform shows the noise floor is somewhat reduced but the speech intelligibility may actually be compromised in sections.

Stage 3 (IIR): The smoothing is visible but cannot recover the speech energy lost in Stage 2. The final output retains the structural problem introduced by the non-stationary noise estimate.

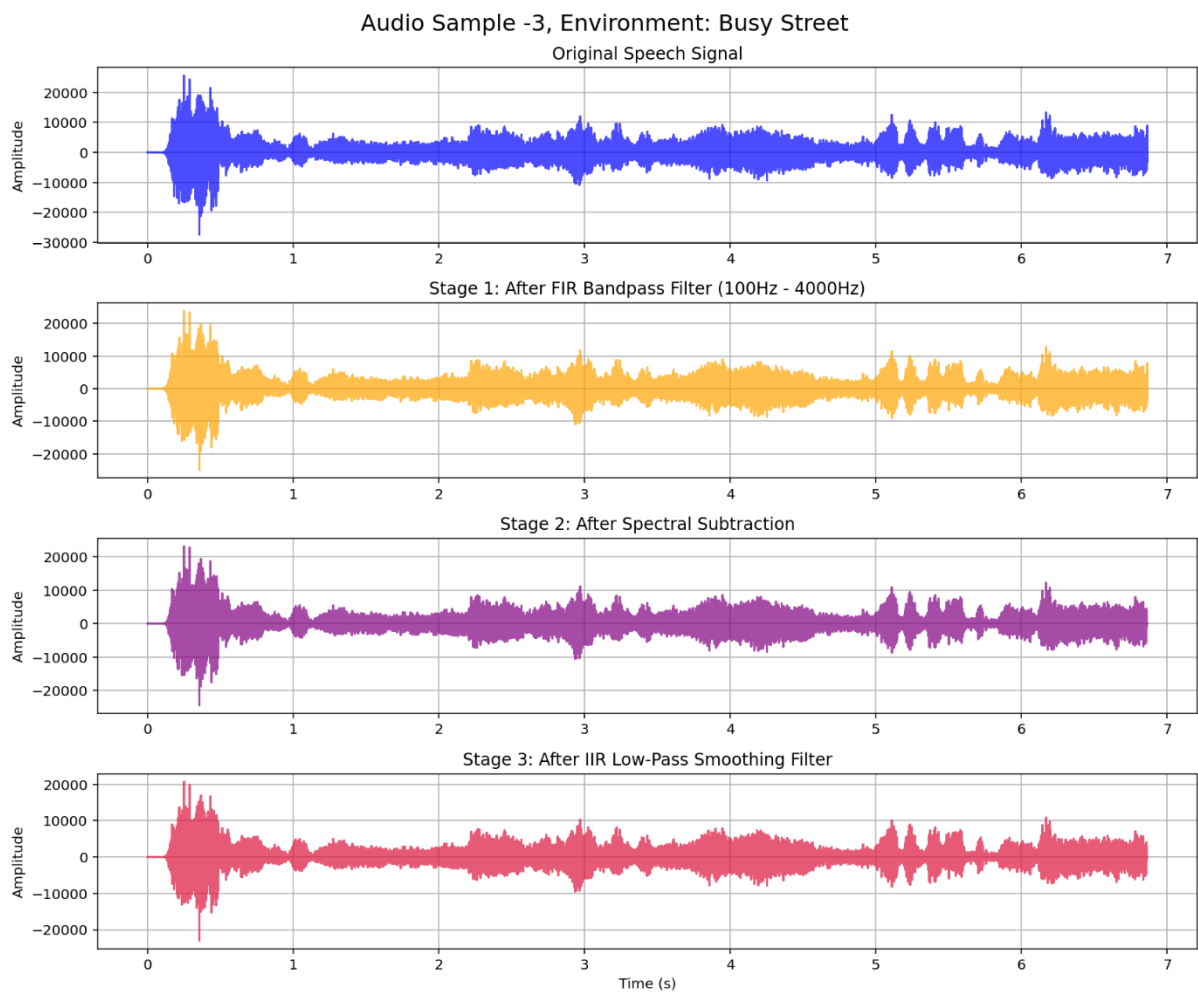


Fig. 3 Original and Filtered Plot of the Sample Speech - 3

Overall: The busy street environment violates the stationarity assumption that the spectral subtraction algorithm depends on. The highly variable traffic; with loud bursts

followed by quieter intervals - means no single noise estimate accurately represents the noise across the full recording. This result clearly demonstrates the boundary of the system's capability and directly validates the constraint stated in the problem formulation: the system is designed for stationary additive noise only.

5.3 Quantitative Analysis:

Stage	Method	Key Parameters	Primary Effect	Known Artifact
FIR	Windowed-Sinc Bandpass	Taps=401, BW=100–4000Hz, Hamming window	Band isolation, noise pre-filtering	200-sample group delay
Spectral Subtraction	STFT Spectral Subtraction	$\alpha=2$, $\beta=0.08$, frame=1024, hop=512	Noise suppression in frequency domain	Residual musical noise
IIR	1 st order RC low pass	fc=3500Hz, backward Euler	Amplitude smoothing, artifact damping	Mild high-freq. roll off

5.4 Verification of Objectives: The study involved an in-depth exploration of FIR and IIR digital filters, along with Short-Time Fourier Transform (STFT) and spectral subtraction techniques, which fulfilled the Objectives 1 and 2. Furthermore, a comprehensive complex engineering investigation was conducted, involving problem formulation, system design, practical implementation, and critical performance analysis. The investigation addressed the challenges associated with signal processing and evaluated the effectiveness of the implemented techniques, thereby satisfying the Objectives 3 and 4.

6. Conclusion: This project successfully designed, implemented, and tested a multi-stage speech enhancement pipeline combining three complementary DSP techniques; FIR bandpass filtering, STFT-based spectral subtraction, and IIR low-pass smoothing and tested against three real-world audio environments: home, beside highway, and busy street.

The results demonstrated that the system performs effectively when the noise is stationary, additive, and broadband. Sample 1 (home/fan) and Sample 2 (highway) both showed clear noise floor reduction with well-preserved speech structure, confirming the pipeline achieves its design objectives. Sample 3 (busy street) exposed the fundamental limitation - non-stationary

noise with varying amplitude caused the noise estimator to over-subtract speech energy in quieter segments, validating the boundary conditions stated in the problem formulation.

Each stage contributed a distinct role: the FIR filter removed out-of-band noise below 100 Hz and above 4000 Hz; spectral subtraction suppressed the in-band stationary noise in the frequency domain; and the IIR filter provided final smoothing of residual artifacts. Together, the three stages form a coherent and computationally lightweight enhancement pipeline built entirely from first principles.

6.1 Comparison with Alternative Methods:

Method	Pros	Cons	Why Not Chosen
Wiener Filter	Optimal in MSE sense	Requires SNR knowledge	SNR not known a priori
LMS Adaptive Filter	Tracks non-stationary noise	Requires reference noise signal	Single-channel input only
Wavelet Denoising	Multi-resolution, no FFT frame	Threshold design is non-trivial	Added complexity without clear gain
DNN-based Enhancement	State-of-the-art quality	Requires large training dataset, GPU	Out of scope for this lab setting
Spectral Subtraction	No reference needed, interpretable	Musical noise artifact	Chosen — best fit for constraints

6.2 Limitations:

- Noise estimated once from first 12 frames fails when noise changes over time
- Musical noise artifacts cannot be fully eliminated with basic spectral subtraction
- Non-stationary noise (horns, bursts, background speech) is outside system scope.
- Parameters α , β , and $f_{c_{iir}}$ are fixed; no automatic adaptation to different noise levels
- Offline only, cannot operate in real-time streaming mode

6.3 Future Development:

- Implement an adaptive noise estimator (e.g., minimum statistics or MMSE-based) to handle non-stationary noise environments.

- Replace simple spectral subtraction with MMSE-STSA (Minimum Mean Square Error Short-Time Spectral Amplitude) estimation for reduced musical noise.
- Add a SNR estimator to automatically tune α and β rather than using fixed values.
- Integrate a Perceptual Weighting Filter that emphasizes frequency bands most critical to human speech intelligibility.

6.4 Application:

- VoIP and teleconferencing - suppression of office HVAC and fan noise
- Hearing aids - reducing constant ambient noise in indoor environments
- ASR preprocessing - improving automatic speech recognition accuracy in noisy conditions
- Podcast and broadcast post-production - removing room noise and microphone hum
- Forensic audio enhancement - cleaning recorded evidence for intelligibility analysis
- Embedded communication devices - intercoms and walkie-talkies in steady-noise environments

7. Appendix:

7.1 Complex Engineering Activities:

A1 (Range of Resources): The project required diverse resources engineering knowledge, audio recording equipment for three environments (home, highway, busy street), Python programming tools, and research literature on spectral subtraction methods.

A2 (Level of Interaction): Significant interaction between conflicting technical requirements was encountered and resolved - particularly the trade-off between noise suppression (α), musical noise (β), and speech intelligibility (IIR cutoff), requiring iterative parameter adjustment.

A3 (Innovation): The project creatively combines three independent DSP principles - FIR design, STFT-domain spectral subtraction, and IIR smoothing - into a single coherent pipeline, applying each technique from first principles without pre-built library functions.

A5 (Familiarity): The project extends beyond standard DSP coursework into the domain of speech acoustics and real-world noise environments. Understanding the stationarity behavior of home, highway, and traffic noise required applying principles-based approaches to an unfamiliar problem context.

7.2 Project Resources: Developed system Source code and the tested audio sample in three different environments also output of those sample after speech enhancement are available in the following Google drive location. The QR code on the right contain the link of the drive file as well.

https://drive.google.com/drive/folders/10HAkdIMMUKrnHnbJ0ZmSqCppioVBR_jR



7.3 Python Source Code:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.io import wavfile

file_path = 'Sample_1_Home.wav'

sample_rate, data = wavfile.read(file_path)

if len(data.shape) > 1:

    print("Converting stereo to mono...")

    data = data[:, 0]

print(f"Sample Rate: {sample_rate} Hz")

print(f"Total Samples: {len(data)}")

# STAGE 1: FIR Bandpass Filter

def create_fir_bandpass(taps, f_low, f_high, fs):

    if taps % 2 == 0:

        taps += 1

    M = taps - 1

    n = np.arange(-M//2, M//2 + 1)

    w1 = 2 * np.pi * f_low / fs

    w2 = 2 * np.pi * f_high / fs
```

```

h = np.zeros(taps)

for i, idx in enumerate(n):

    if idx == 0:

        h[i] = (w2 - w1) / np.pi

    else:

        h[i] = (np.sin(w2 * idx) - np.sin(w1 * idx)) / (np.pi * idx)

window = 0.54 + 0.46 * np.cos(2 * np.pi * n / M)

h_windowed = h * window

return h_windowed

print("Designing FIR Filter...")

num_taps = 401

low_cut = 100

high_cut = 4000

fir_coefficients = create_fir_bandpass(num_taps, low_cut, high_cut,
sample_rate)

print("Applying FIR Filter to audio...")

filtered_data = np.convolve(data, fir_coefficients, mode='same')

filtered_data_int16 = np.int16(np.clip(filtered_data, -32768, 32767))

output_path_1 = 'output_audio_1_fir.wav'

wavfile.write(output_path_1, sample_rate, filtered_data_int16)

print(f"Stage 1 audio saved to {output_path_1}")

# STAGE 2: Spectral Subtraction (From Scratch)

print("Applying Spectral Subtraction...")

frame_size = 1024

hop_size = 512

analysis_window = np.hamming(frame_size)

```

```

alpha = 4.0

beta = 0.08

num_noise_frames = 12

noise_magnitudes = []

for f in range(num_noise_frames):

    start = f * hop_size

    end = start + frame_size

    if end <= len(filtered_data):

        noise_frame = filtered_data[start:end] * analysis_window

        noise_fft = np.fft.fft(noise_frame)

        noise_magnitudes.append(np.abs(noise_fft))

avg_noise_magnitude = np.mean(noise_magnitudes, axis=0)

spec_sub_output = np.zeros_like(filtered_data, dtype=float)

window_weight_sum = np.zeros_like(filtered_data, dtype=float)

for start_idx in range(0, len(filtered_data) - frame_size + 1, hop_size):

    end_idx = start_idx + frame_size

    speech_frame = filtered_data[start_idx:end_idx] * analysis_window

    speech_fft = np.fft.fft(speech_frame)

    magnitude = np.abs(speech_fft)

    phase = np.angle(speech_fft)

    subtracted_magnitude = magnitude - (alpha * avg_noise_magnitude)

    floor_mask = subtracted_magnitude < (beta * magnitude)

    subtracted_magnitude[floor_mask] = beta * magnitude[floor_mask]

    reconstructed_fft = subtracted_magnitude * np.exp(1j * phase)

    clean_time_frame = np.real(np.fft.ifft(reconstructed_fft))

    spec_sub_output[start_idx:end_idx] += clean_time_frame * analysis_window

    window_weight_sum[start_idx:end_idx] += analysis_window ** 2

```

```

window_weight_sum[window_weight_sum == 0] = 1.0

spec_sub_output /= window_weight_sum

spec_sub_output_int16 = np.int16(np.clip(spec_sub_output, -32768, 32767))

output_path_2 = 'output_audio_2_specsub.wav'

wavfile.write(output_path_2, sample_rate, spec_sub_output_int16)

print(f"Stage 2 audio saved to {output_path_2}")


# STAGE 3: IIR Low-Pass Smoothing Filter

print("Applying IIR Low-Pass Filter...")

fc_iir = 3500.0

dt = 1.0 / sample_rate

tau = 1.0 / (2 * np.pi * fc_iir)

alpha_iir = dt / (tau + dt)

iir_output = np.zeros_like(spec_sub_output)

previous_y = 0.0

for n in range(len(spec_sub_output)):

    current_y = (alpha_iir * spec_sub_output[n]) + ((1.0 - alpha_iir) *
previous_y)

    iir_output[n] = current_y

    previous_y = current_y

final_smoothed_int16 = np.int16(np.clip(iir_output, -32768, 32767))

output_path_3 = 'output_audio_3_final.wav'

wavfile.write(output_path_3, sample_rate, final_smoothed_int16)

print(f"Final completely smoothed audio saved to {output_path_3}")


# Verification: Plot the complete pipeline output

time = np.arange(len(data)) / sample_rate

plt.figure(figsize=(12, 10))

```

```

plt.suptitle("Audio Sample -1, Environment: Home", fontsize = 16)

# Plot 1: Original Audio
plt.subplot(4, 1, 1)
plt.plot(time, data, color='blue', alpha=0.7)
plt.title('Original Speech Signal')
plt.ylabel('Amplitude')
plt.grid(True)

# Plot 2: After FIR Bandpass Filter
plt.subplot(4, 1, 2)
plt.plot(time, filtered_data_int16, color='orange', alpha=0.7)
plt.title('Stage 1: After FIR Bandpass Filter (100Hz - 4000Hz)')
plt.ylabel('Amplitude')
plt.grid(True)

# Plot 3: After Spectral Subtraction
plt.subplot(4, 1, 3)
plt.plot(time, spec_sub_output_int16, color='purple', alpha=0.7)
plt.title('Stage 2: After Spectral Subtraction')
plt.ylabel('Amplitude')
plt.grid(True)

# Plot 4: After IIR Smoothing Low-Pass Filter
plt.subplot(4, 1, 4)
plt.plot(time, final_smoothed_int16, color='crimson', alpha=0.7)
plt.title('Stage 3: After IIR Low-Pass Smoothing Filter')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid(True)

plt.tight_layout()
plt.show()

```